Exam-Slot Assignment with QAOA and Simulated Annealing: Part II

Group G3: Alef Keuffer, Diogo Ferreira, Luis Arruda

November 10, 2026

Contents

1 Strategy

2 EMF chain

3 BESSER chain

4 Validation

5 Limitations

A Rule-to-reference traceability

B Full results

C Metamodels

D Test instances

E Reproducing the results

1 Strategy

Terms are those of the EMF chain's glossary, `emf/GLOSSARY.md`.[1]

A Problem lists students, the exams each of them sits, and a fixed number of batches; all exams of a batch take place at once. Two exams that share a student are in Conflict and must not share a batch [7, §5.2, Algorithm 1]. Each exam carries a Weight, its priority: the cost of leaving it unscheduled.

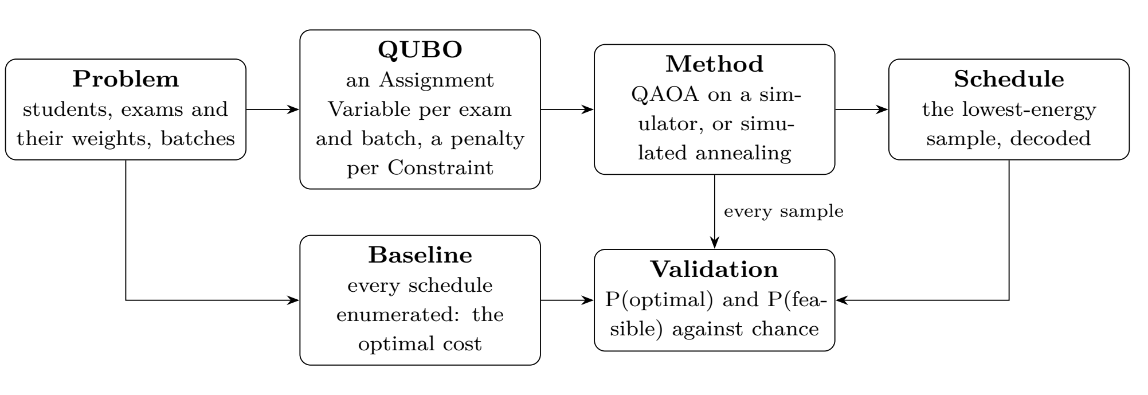
Both chains follow Fig. 1. The Problem is formulated as a QUBO over one Assignment Variable x_{ik} per exam i and batch k, 1 when exam i is in batch k [7, §5.3, Eq. (4)], with a penalty per Constraint. A Method samples the QUBO, and the lowest-energy sample is decoded into a Schedule. Independently of the QUBO, a Baseline enumerates every schedule to find the optimal cost, and every sample, not only the best, is judged against it (Section 4).

The chains implement two formulations. The paper's [7, §5.3, Eqs. (5)–(10)], which the BESSER chain implements, places each exam in exactly one batch and has penalties only: a sample of energy zero is a schedule without conflicts, and nothing ranks one exam above another. The EMF chain implements a weighted one (`emf/docs/adr/0009`): each exam in at most one batch [4, §4.2, Eq. (30)], with every scheduled exam earning its weight, a weighted set packing [2, §4.2]:

$$H = -\sum_i w_i \sum_k x_{ik} + A\Big(\sum_i \sum_{k<l} x_{ik}x_{il} + \sum_{(i,j)\ \text{in Conflict}} \sum_k x_{ik}x_{jk}\Big).$$

With the Penalty Coefficient A above every weight, leaving an exam unscheduled always costs less than breaking a Constraint [4, §4.2], so the optimum is a feasible Schedule of least Cost, the total weight of its Unscheduled Exams. The two optima differ wherever not every exam fits. On four mutually conflicting exams in three batches (BESSER's `none`, EMF's `clique`), the paper's formulation accepts one Conflict, two exams in one batch; the weighted one leaves the lightest exam, Statistics, unscheduled at Cost 1.

The paper samples its QUBO by hybrid quantum annealing on D-Wave hardware [7]; the reference notebook calls D-Wave's quantum annealer and its hybrid solver [5, cells 16–17]. Both chains sample it instead by simulated annealing, its classical counterpart, as the notebook also does [5, cell 15]: it needs no D-Wave account, so CI reproduces every run, and it gives QAOA, which a simulator limits to small instances, a yardstick on the same QUBO. Quantum annealing itself is issue #23.



Problem
students, exams and their weights, batches
QUBO
an Assignment Variable per exam and batch, a penalty per Constraint
Method
QAOA on a simulator, or simulated annealing
Schedule
the lowest-energy sample, decoded
every sample
Baseline
every schedule enumerated: the optimal cost
Validation
P(optimal) and P(feasible) against chance

Figure 1: The strategy of both chains: model the Problem, formulate it as a QUBO, sample it by a Method, decode the lowest-energy sample into a Schedule, and judge every sample against a Baseline that enumerates every schedule.

2 EMF chain

The EMF chain turns one `.exam` file into a Python script in three model-driven steps, run headless by an Eclipse application; the script then solves the Problem and validates what it finds. The models parsed from the file, and the Conflict Graph, are checked against their metamodels' multiplicities and OCL invariants (Section C) before the next step uses them.

Xtext. The grammar imports the hand-written Problem and Solver Configuration metamodels, so one `.exam` file holds a Problem (its batches, its students, and each exam with its optional weight, 1 by default, and the students who sit it) and, optionally, its Solver Configuration: the Method, the Method's settings, the seed and the Penalty Coefficient, automatic unless given (Section D).

ATL. An EMFTVM transformation makes the Problem a Conflict Graph [7, §5.2, Fig. 3]. It copies the exams with their weights, creates the M batches, the columns of x [7, §5.3, Eq. (4)], and adds one Conflict for each pair of exams that share a student, the paper's contradiction matrix C [7, §5.2, Algorithm 1, Eq. (3)].

Acceleo. A template generates `solve_<name>.py` from the Conflict Graph and the Solver Configuration: both as Python data, then the notebook's pyqubo construction [5, cells 11–13] [6], made the weighted set packing of Section 1 (`emf/docs/adr/0007`). Each Constraint is labelled in domain terms, `at-most-one-batch-<exam>` or `conflict-<exam>-<exam>-batch-<k>`, so a broken one names its exams. pyqubo compiles H into the QUBO and its Ising form, so the chain models neither.

The runtime. The generated script holds only what depends on the models; it imports the hand-written `emf/python/examschedule.py`, which holds the rest (`emf/docs/adr/0005`). It sets A to the largest weight plus one, or rejects an explicit A not above every weight [2, §4, Comment on the Scalar Penalty P], and samples by the Method chosen. Simulated annealing runs dwave-neal on the compiled model, as the notebook does [5, cell 15]. QAOA [1] builds the QAOA Circuit from the Ising form, with p alternating cost and mixer layers on the uniform superposition [1, Eqs. (2)–(6)], runs it on Qiskit Aer [3], and lets a classical optimiser tune the $2p$ angles to minimise the expected energy (Farhi et al. maximise theirs) [1, Eqs. (7)–(8)], starting on the annealing path [1, §VI]. The runtime then decodes the lowest-energy sample into a Schedule model [7, §5.5], runs the Baseline, and writes the run's results, from which Section B is generated.

Every ATL rule, template section and runtime step cites its reference in a comment beside it, as `[Key locator]`; Table 2 is generated from those comments.

3 BESSER chain

Missing: The BESSER chain. Waits on the BESSER rework.
T1 (Problem → QUBO, rules R1–R6), T2 (QUBO → QAOA circuit in BESSER's quantum metamodel, rules Q1–Q7), T3 (BESSER's `QiskitGenerator`) and the mapping of samples back to Schedules, with the reference of each rule. BESSER's code cites the paper as `[Zhou]`; with `[Paper …]` tags, Section A could list its rules too.

4 Validation

Missing: How the chains are validated. Waits on the team.
What every run measures: the Schedule found, whether it is feasible and optimal against the Baseline, P(optimal) and P(feasible) over every shot or read, and the odds of a uniformly random bit string, the bar a Method must clear. The EMF chain also checks its formulation in every run: a sample's energy is minus the weight it places exactly when it breaks no Constraint. Each chain is measured on its own instances (Section D); every run is in Section B.

Table 1: P(optimal) of the EMF chain on each test instance (Section D) by Method, against a uniformly random bit string's; in brackets, how many times chance. Solver Configurations in Table 3; from CI run 42879932659.

Instance	Variables	Random	QAOA	Simulated annealing
path	8	0.0078	0.055 (7×)	0.960 (123×)
cycle	15	0.00092	0.033 (36×)	1.000 (1092×)
clique	12	0.0015	0.044 (30×)	0.980 (669×)
semester	18	0.000046	0.0017 (37×)	0.980 (21408×)

Missing: The BESSER chain's table, P(optimal) against chance on its instances. Waits on the BESSER rework.
From `besser/output/<instance>/6_validation.json` once the chain settles.

5 Limitations

Missing: The limitations. Waits on the team.

- QAOA is far behind simulated annealing (Table 1), and only simulated, on at most 18 qubits.
- QAOA starts from a fixed ramp of angles on the annealing path [1, §VI], not set by the Solver Configuration. From there a deeper circuit raises P(feasible) on path and cycle, but there P(optimal) stays between 0.028 and 0.062 whatever the depth or optimiser (Tables 3 and 5): the extra feasible schedules leave an exam out.
- The simulated annealing is classical: no quantum annealer was used (issue #23).
- The Baseline enumerates every schedule, so validation stops at about 10^6 schedules.
- The Schedule the generated script writes is not validated against its metamodel's OCL.
- The BESSER chain's own.

References

[1] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. *A Quantum Approximate Optimization Algorithm*. 2014. arXiv: `1411.4028v1` (cited in § 2, § 5, § A).

[2] Fred Glover, Gary Kochenberger, and Yu Du. *Quantum Bridge Analytics I: A Tutorial on Formulating and Using QUBO Models*. Published in 4OR 17 (2019) 335–371. 2019. arXiv: `1811.11538v6` (cited in § 1, § 2, § A).

[3] Ali Javadi-Abhari et al. *Quantum computing with Qiskit*. 2024. arXiv: `2405.08810` (cited in § 2, § E).

[4] Andrew Lucas. "Ising formulations of many NP problems". In: *Frontiers in Physics* 2 (2014), p. 5. DOI: `10.3389/fphy.2014.00005`. arXiv: `1302.5843v3` (cited in § 1, § A).

[5] *`QuantumExamScheduling.ipynb`*. The reference notebook of the paper's formulation, in the repository as `emf/QuantumExamScheduling.ipynb`; cells are numbered by their execution counts (cited in § 1, § 2, § A).

[6] Mashiyat Zaman, Kotaro Tanahashi, and Shu Tanaka. "PyQUBO: Python Library for Mapping Combinatorial Optimization Problems to QUBO Form". In: *IEEE Transactions on Computers* 71.4 (2022), pp. 838–850. DOI: `10.1109/TC.2021.3063618` (cited in § 2, § E).

[7] Ziyu Zhou et al. "Hybrid quantum annealing for large-scale exam scheduling: Validation in real-world educational scenarios". In: *Applied Soft Computing* 184 (2025), p. 113756. DOI: `10.1016/j.asoc.2025.113756` (cited in § 1, § 2, § A).

A Rule-to-reference traceability

Table 2: Every ATL rule, template section and runtime step of the EMF chain that implements the formulation or QAOA, with the reference its comment cites, read from the code.

Unit	What it implements, as its comment says
ATL, `problem2conflictgraph.atl`: Problem → Conflict Graph	
`rule Problem2ConflictGraph`	[7, §5.2, Fig. 3]: the conflict graph G, exams as nodes and conflicts as edges.
`rule Problem2ConflictGraph`	[7, §5.3, Eq. (4)]: X has only M columns, so the M batches bound the schedule.
`rule Exam2Exam`	[7, §5.2, Fig. 3]: each exam is a node of G.
`rule Exam2Exam`	[2, §4.2]: the weight w_j of the set packing objective, the exam's priority (ADR-0009).
`rule NewBatch`	[7, §5.3, Eq. (4)]: batch k is column k of X, k = 1..M.
`helper sharesStudentWith`	[7, §5.2, Algorithm 1, Eq. (3)]: C[i,j] = 1 iff some student sits both exam i and exam j.
`lazy rule Conflict`	[7, §5.2, Fig. 3]: one edge of G per pair with C[i,j] = 1, emitted once, in problem order.
Acceleo, `main.mtl`: Conflict Graph and Solver Configuration → `solve_<name>.py`	
`CONFLICTS`	[7, §5.2, Algorithm 1, Eq. (3)] [5, cell 7]: C, the contradiction matrix, as pairs (i, j), i < j.
`x`	[7, §5.3, Eq. (4)] [5, cell 11]: x[i, k] = 1 iff exam i is in batch k + 1
`constraint1`	[2, §4, known penalties, row 5] [4, §4.2, Eq. (30)]: each exam in at most one batch, the paper's exactly-one [7, §5.3, Eqs. (5)–(7)] relaxed (ADR-0009). With one batch there is no pair to forbid.
`constraint2`	[7, §5.3, Eqs. (8)–(9)] [5, cell 12]: conflicting exams never share a batch, the penalty x[i, k] x[j, k] of [2, §4, known penalties, row 1].
`profit`	[2, §4.2] [4, §4.2, Eq. (31)]: scheduling exam i earns its weight; an unscheduled exam costs it
`A, H, model`	[7, §5.3, Eq. (10)] [5, cell 13]: H, with lambda = gamma = A and the profit added; A is fed from the Solver Configuration
Python, `examschedule.py`: the runtime the generated scripts call	
`schedule_of`	[7, §5.5]: the batch of each exam read off X, x[i, k] = 1 iff exam i is in batch k + 1.
`penalty`	A: the largest weight plus one unless set. An explicit A must exceed every weight, or breaking a constraint can cost less than leaving an exam unscheduled (ADR-0009). [4, §4.2]: "Choosing B < A ensures that it is never favorable to violate the constraint", per weight here. [2, §4, Comment on the Scalar Penalty P]: P large enough that a hard constraint is never broken.
`anneal`	[5, cell 15]: simulated annealing (neal) on the BQM, then pyqubo decodes every read.
`qaoa`	[1, Eqs. (1)–(8)]: QAOA on the Ising form of the compiled model, simulated on Aer, with C = H and the expectation minimised, where Farhi maximises it.
`qaoa`	[4, §2.3, Eq. (12)]: pyqubo's spins, x = (s + 1)/2, so s = 2x - 1.
`qaoa`	[1, Eq. (5)]: |s>, the uniform superposition, top eigenstate of the mixer B
`qaoa`	[1, Eq. (6)]: p alternating cost and mixer layers.
`qaoa`	[1, Eq. (2)]: cost layer U(C, gamma) = exp(-i gamma C), C = H/scale. [4, §1.1, Eq. (4)]: on a qubit Z = 1 - 2x = -s, so a field h on s acts as -h on Z.
`qaoa`	[1, Eqs. (3)–(4)]: mixer layer U(B, beta) = exp(-i beta B), B = sum_j X__j.
`expected_energy`	[1, Eq. (7)]: F__p, the expectation of the cost in the state, estimated from the shots.
`qaoa`	[1, §VI]: a linear ramp (gamma grows, |beta| shrinks) on the Trotterised adiabatic path from |s>, the top of B, to the top of -C, the minimum of C; with the cost layer exp(-i gamma C), that path has gamma and beta of opposite signs.
`qaoa`	[1, Eq. (8)]: the classical loop optimises the 2p angles.

Missing: The BESSER chain's rules and their references. Waits on the BESSER rework.
Once its code cites `[Key locator]` as the EMF chain's does, `generate.py` can read it too.

B Full results

Table 3: Every run of the EMF chain: each test instance solved by both Methods, with the committed seeds; the Penalty Coefficient A used, the Schedule's cost and Unscheduled Exams, the Baseline's optimal cost, and P(optimal) and P(feasible) against a uniformly random bit string's. From CI run 42879932659.

					Optimal	P(optimal)		P(feasible)	
Instance	Var.	Method	A	Result	cost	run	random	run	random
path	8	QAOA, depth 2, COBYLA, 150 iterations, 2048 shots	4	0, none	0	0.055	0.0078	0.752	0.160
path	8	annealing, 50 reads × 10000 sweeps	4	0, none	0	0.960	0.0078	1.000	0.160
cycle	15	QAOA, depth 2, COBYLA, 150 iterations, 4096 shots	2	0, none	0	0.033	0.00092	0.749	0.012
cycle	15	annealing, 50 reads × 10000 sweeps	2	0, none	0	1.000	0.00092	1.000	0.012
clique	12	annealing, 50 reads × 10000 sweeps	5	1, Statistics	1	0.980	0.0015	1.000	0.018
clique	12	QAOA, depth 2, COBYLA, 150 iterations, 4096 shots	5	1, Statistics	1	0.044	0.0015	0.657	0.018
semester	18	annealing, 50 reads × 10000 sweeps	4	0, none	0	0.980	0.000046	1.000	0.0034
semester	18	QAOA, depth 2, COBYLA, 150 iterations, 4096 shots	4	0, none	0	0.0017	0.000046	0.611	0.0034

Table 4: The Schedule each run of Table 3 found: the exams of each batch.

Instance	Method	Batch: exams	Unscheduled
path	QAOA	1: Calculus, Chemistry; 2: Algebra, Physics	none
path	annealing	1: Calculus, Chemistry; 2: Algebra, Physics	none
cycle	QAOA	1: Calculus; 2: Algebra, Chemistry; 3: Physics, Programming	none
cycle	annealing	1: Chemistry; 2: Calculus, Programming; 3: Algebra, Physics	none
clique	annealing	1: Physics; 2: Calculus; 3: Algebra	Statistics
clique	QAOA	1: Physics; 2: Algebra; 3: Calculus	Statistics
semester	annealing	1: Physics, Programming; 2: Calculus, Databases; 3: Algebra, Chemistry	none
semester	QAOA	1: Calculus, Databases; 2: Algebra, Chemistry; 3: Physics, Programming	none

Table 5: The EMF chain with other QAOA settings, every one reaching the optimal cost. Measured locally on macOS (arm64) with the pinned `emf/requirements.txt`, not in CI, so a rerun on Linux may differ in the second digit; copied by hand from `emf/CONTEXT.md`. COBYLA stops well before 150 iterations (29 to 54 evaluations); a Powell iteration is a line search over every angle (398 evaluations on cycle).

Instance	Variables	Method	P(optimal)	P(feasible)
path	8	QAOA, depth 1, COBYLA, 150 iterations, 2048 shots	0.031	0.723
path	8	QAOA, depth 3, COBYLA, 150 iterations, 2048 shots	0.062	0.881
path	8	QAOA, depth 2, Nelder-Mead, 150 iterations, 2048 shots	0.049	0.796
cycle	15	QAOA, depth 1, COBYLA, 150 iterations, 4096 shots	0.028	0.517
cycle	15	QAOA, depth 3, COBYLA, 150 iterations, 4096 shots	0.047	0.865
cycle	15	QAOA, depth 2, Powell, 150 iterations, 4096 shots	0.036	0.809

Missing: The BESSER chain's full results. Waits on the BESSER rework.
Every instance's metrics, from `besser/output/summary.md` or its JSONs.

C Metamodels

Containment is drawn with a diamond at the container, a reference with an open arrow; *inv* names an OCL invariant.

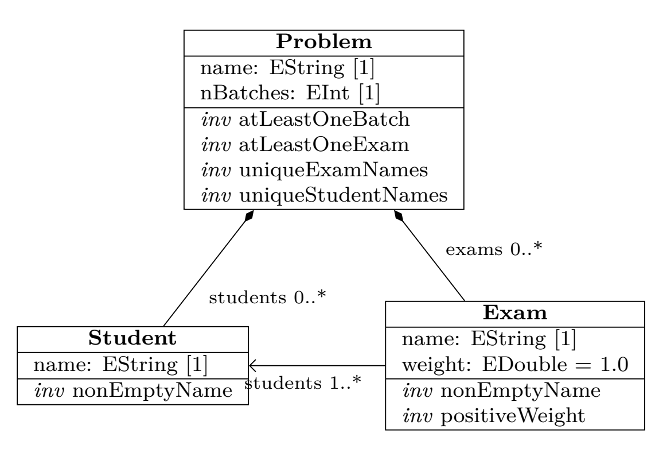


Figure 2: The problem metamodel, drawn from `problem.ecore`.

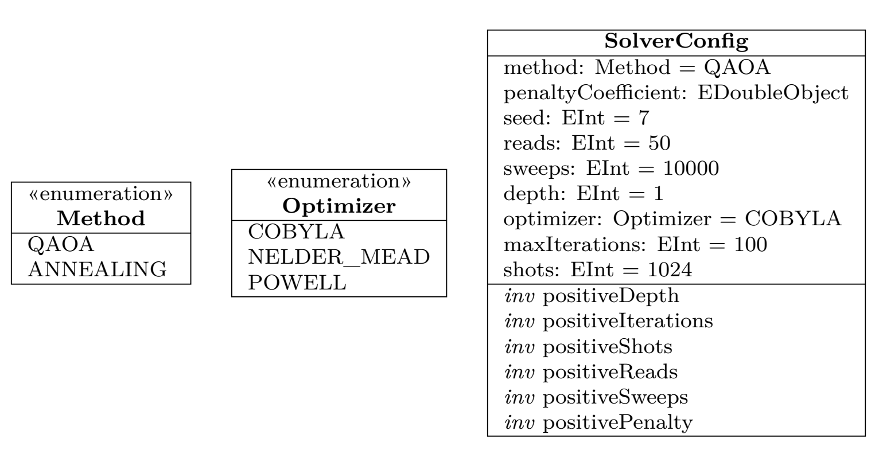


Figure 3: The solverconfig metamodel, drawn from `solverconfig.ecore`.

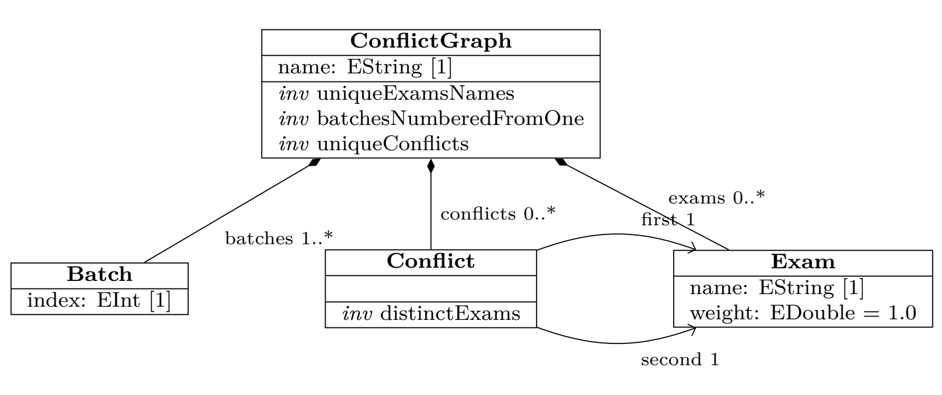


Figure 4: The conflictgraph metamodel, drawn from `conflictgraph.ecore`.

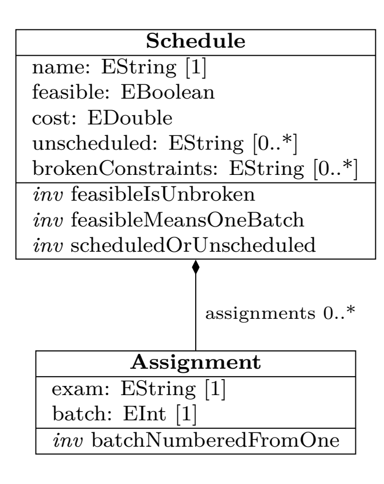


Figure 5: The schedule metamodel, drawn from `schedule.ecore`.

Missing: The BESSER chain's metamodels. Waits on the BESSER rework.
Part I's diagrams (`docs/imgs/problem-dsl-01.png`, `docs/imgs/solution-dsl-01.png`) once BESSER settles, with its Solver Configuration and quantum metamodels.

D Test instances

The EMF chain's instances, `emf/instances/`. Each has a twin, `<n>-<name>-qaoa.exam` or `<n>-<name>-annealing.exam`, holding the same problem solved by the other Method.

`1-path.exam`

```
// Q16 (i): four exams whose conflicts form a path, two batches. Feasible (a path is 2-colourable).
problem path batches 2
students Ana Rui Eva
exam Algebra   weight 2 { Ana }
exam Calculus  weight 3 { Ana Rui }
exam Physics   weight 1 { Rui Eva }
exam Chemistry weight 2 { Eva }

solver { method QAOA depth 2 optimizer COBYLA iterations 150 shots 2048 seed 7 }
```

`2-cycle.exam`

```
// Q16 (ii): five exams whose conflicts form a 5-cycle, three batches. Feasible (needs 3 batches).
// No weights: every exam weighs 1, so the schedule places as many exams as it can.
problem cycle batches 3
students Ana Rui Eva Ivo Lia
exam Algebra { Ana Lia }
exam Calculus { Ana Rui }
exam Physics { Rui Eva }
exam Chemistry { Eva Ivo }
exam Programming { Ivo Lia }

solver { method QAOA depth 2 optimizer COBYLA iterations 150 shots 4096 seed 7 }
```

`3-clique.exam`

```
// Q16 (iii): four exams that all conflict, three batches. No schedule places them all: the
// lowest-weight exam, Statistics, is left unscheduled, at cost 1 (ADR-0009).
problem clique batches 3
students Ana Rui Eva
exam Algebra    weight 4 { Ana Rui Eva }
exam Calculus   weight 3 { Ana Rui Eva }
exam Physics    weight 2 { Ana Rui Eva }
exam Statistics weight 1 { Ana Rui Eva }

solver { method annealing reads 50 sweeps 10000 seed 7 }
```

`4-semester.exam`

```
// Q16 (iv): six exams of a semester, three batches. Feasible.
problem semester batches 3
students Ana Rui Eva Ivo Lia Tom
exam Algebra     weight 3 { Ana Rui }
exam Calculus    weight 3 { Rui Eva Tom }
exam Physics     weight 2 { Ana Eva }
exam Chemistry   weight 2 { Ivo Lia }
exam Programming weight 1 { Lia Tom }
exam Databases   weight 1 { Ivo Ana }

solver { method annealing reads 50 sweeps 10000 seed 7 }
```

Missing: The BESSER chain's instances. Waits on the BESSER rework.
`besser/instances/`: tiny, many, none, medium and limit.

E Reproducing the results

- The EMF chain: `cd emf && ./build.sh` (JDK 21+, Python 3.12) builds the chain and runs it on every instance, writing `emf/out/<name>/results_<name>.json`. The Python packages, pyqubo [6], Qiskit and Qiskit Aer [3] and dwave-neal among them, are pinned in `emf/requirements.txt`. CI runs the same on every push (`.github/workflows/emf.yml`) and uploads the results as the `emf-results` artifact.
- This report: `docs/report_2/build.sh` (TeX Live, `gh`) fetches those results from CI, generates its tables, Section A and the diagrams of Section C, and runs latexmk; it does not run the chains.
- The LLM logs of Part II: `docs/llm_usage_2.md`, with the session records it links in `docs/assets/`, and `besser/docs/llm_usage_2.md`.

Missing: How to reproduce the BESSER chain's results. Waits on the BESSER rework.
`besser/README.md`'s commands, once the chain settles.

---

[1] BESSER's metamodels, from Part I, call a Batch a *TimeSlot* (or slot) and a feasible Schedule *valid*.